CS24352 • PRACTICAL LAB MANUAL

Natural Language Processing Practical Lab Manual

Complete 10 Practical Experiments with Python, PyTorch, Transformers, Word2Vec & BLEU

Author: Shaswat Raj (BIT Mesra CSE) **Academic Scheme:** NEP 2024–25 (5th Sem) **Credits:** 1.5 Practical Credits

Experiment 1: End-to-End NLP Text Preprocessing Pipeline

Objective: Implement a complete text normalization pipeline in Python: Regex tokenization, sentence boundary detection, punctuation and stop-word filtering, Porter stemming, and WordNet lemmatization.

```
import re

class TextPreprocessor:
    def __init__(self):
        self.stop_words = {'the', 'is', 'at', 'which', 'on', 'a', 'an', 'and', 'in', 'to', 'for', 'of'}

    def tokenize(self, text):
        # Match alphanumeric word tokens
        return re.findall(r'\b\w+\b', text.lower())

    def remove_stopwords(self, tokens):
        return [w for w in tokens if w not in self.stop_words]

    def simple_stemmer(self, word):
        # Suffix stripping heuristics
        if word.endswith('ing') and len(word) > 5: return word[:-3]
        if word.endswith('ly') and len(word) > 4: return word[:-2]
        if word.endswith('ed') and len(word) > 4: return word[:-2]
        if word.endswith('es') and len(word) > 4: return word[:-2]
        if word.endswith('s') and len(word) > 3: return word[:-1]
        return word

    def process(self, text):
        tokens = self.tokenize(text)
        filtered = self.remove_stopwords(tokens)
        stems = [self.simple_stemmer(w) for w in filtered]
        return {'Original Tokens': tokens, 'Filtered': filtered, 'Stems': stems}

pipe = TextPreprocessor()
sample_text = "Natural Language Processing is transforming how computers are understanding human languages!"
res = pipe.process(sample_text)
print("Processed Pipeline Results:\n", res)
```

Experiment 2: Byte-Pair Encoding (BPE) Subword Tokenizer from Scratch

Objective: Implement the Byte-Pair Encoding (BPE) subword tokenization algorithm (used in GPT-2/3/4) by iteratively finding and merging the most frequent adjacent symbol pairs.

```
from collections import Counter, defaultdict

def get_stats(vocab):
    pairs = defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[(symbols[i], symbols[i + 1])] += freq
    return pairs

def merge_vocab(pair, v_in):
    v_out = {}
    bigram = ' '.join(pair)
    replacement = ''.join(pair)
    for word in v_in:
        w_out = word.replace(bigram, replacement)
        v_out[w_out] = v_in[word]
    return v_out

# Initial character-level vocabulary with end-of-word tag ''
vocab = {'l o w ': 5, 'l o w e r ': 2, 'n e w e s t ': 6, 'w i d e s t ': 3}
num_merges = 6

print("Initial Vocabulary:", vocab)
for i in range(num_merges):
    pairs = get_stats(vocab)
    if not pairs: break
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    print(f"Merge {i+1}: Best Pair {best} (Freq = {pairs[best]}) → Merged Vocab: {vocab}")
```

Experiment 3: Minimum Edit Distance (Wagner-Fischer Dynamic Programming)

Objective: Implement the Wagner-Fischer dynamic programming algorithm in Python to compute the Minimum Edit (Levenshtein) Distance with insertion, deletion, and substitution costs.

```
def min_edit_distance(source, target, ins_cost=1, del_cost=1, sub_cost=2):
    m, n = len(source), len(target)
    # DP table of dimensions (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1): dp[i][0] = i * del_cost
    for j in range(n + 1): dp[0][j] = j * ins_cost

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if source[i - 1] == target[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] # Zero cost for match
            else:
                dp[i][j] = min(
                    dp[i - 1][j] + del_cost,       # Deletion
                    dp[i][j - 1] + ins_cost,       # Insertion
                    dp[i - 1][j - 1] + sub_cost    # Substitution
                )
    return dp[m][n]

src, tgt = "intention", "execution"
dist = min_edit_distance(src, tgt)
print(f"Minimum Edit Distance between '{src}' and '{tgt}' = {dist}")
```

Experiment 4: Statistical N-Gram Language Modeling & Perplexity Evaluation

Objective: Implement Bigram and Trigram Language Models with Laplace Add-1 smoothing and compute test set cross-entropy Perplexity metric ($PP(W) = \sqrt[N]{\prod \frac{1}{P(w_i|w_{i-1})}}$).

```
from collections import Counter, defaultdict
import math

class BigramLanguageModel:
    def __init__(self):
        self.unigrams = Counter()
        self.bigrams = Counter()
        self.vocab = set()

    def train(self, corpus):
        for sentence in corpus:
            tokens = ['<s>'] + sentence.lower().split() + ['</s>']
            for i in range(len(tokens) - 1):
                self.unigrams[tokens[i]] += 1
                self.bigrams[(tokens[i], tokens[i+1])] += 1
                self.vocab.add(tokens[i])
            self.unigrams[tokens[-1]] += 1
            self.vocab.add(tokens[-1])

    def probability(self, w1, w2):
        # Laplace Add-1 Smoothing: (Count(w1, w2) + 1) / (Count(w1) + |V|)
        count_bi = self.bigrams.get((w1, w2), 0)
        count_uni = self.unigrams.get(w1, 0)
        return (count_bi + 1) / (count_uni + len(self.vocab))

    def perplexity(self, test_sentence):
        tokens = ['<s>'] + test_sentence.lower().split() + ['</s>']
        log_prob = 0.0
        n = len(tokens) - 1
        for i in range(n):
            p = self.probability(tokens[i], tokens[i+1])
            log_prob += math.log2(p)
        return 2 ** (-log_prob / n)

corpus = ["the cat sat on the mat", "the dog sat on the rug", "cats and dogs are great"]
lm = BigramLanguageModel()
lm.train(corpus)
print("Bigram Probability P(sat | cat):", round(lm.probability('cat', 'sat'), 4))
print("Test Sentence Perplexity:       ", round(lm.perplexity("the cat sat on the rug"), 2))
```

Experiment 5: Part-of-Speech (POS) Tagging with Hidden Markov Models (Viterbi)

Objective: Implement the dynamic programming Viterbi decoding algorithm in Python for HMM POS Tagging, finding the optimal hidden tag sequence $T^* = \arg\max \prod P(w_i \mid t_i) P(t_i \mid t_{i-1})$.

```
import numpy as np

def viterbi_pos_tagger(sentence, states, start_p, trans_p, emit_p):
    words = sentence.split()
    n = len(words)
    k = len(states)
    V = np.zeros((k, n))
    backpointer = np.zeros((k, n), dtype=int)

    # Initialization (t = 0)
    for s in range(k):
        V[s, 0] = start_p[s] * emit_p[s].get(words[0], 1e-6)

    # Recursion (t = 1 to n-1)
    for t in range(1, n):
        for s in range(k):
            prob_candidates = [V[prev_s, t - 1] * trans_p[prev_s, s] * emit_p[s].get(words[t], 1e-6)
                               for prev_s in range(k)]
            V[s, t] = max(prob_candidates)
            backpointer[s, t] = np.argmax(prob_candidates)

    # Termination & Backtracking
    best_last_state = np.argmax(V[:, n - 1])
    best_path = [best_last_state]
    for t in range(n - 1, 0, -1):
        best_path.insert(0, backpointer[best_path[0], t])

    return [states[s] for s in best_path]

states = ['NOUN', 'VERB', 'ADJ']
start_p = [0.6, 0.2, 0.2]
trans_p = np.array([[0.3, 0.6, 0.1], [0.5, 0.1, 0.4], [0.7, 0.1, 0.2]])
emit_p = [
    {'time': 0.5, 'flies': 0.2, 'arrow': 0.3},
    {'time': 0.1, 'flies': 0.7, 'arrow': 0.2},
    {'time': 0.4, 'flies': 0.1, 'arrow': 0.5}
]

tags = viterbi_pos_tagger("time flies", states, start_p, trans_p, emit_p)
print("Optimal Viterbi POS Tag Sequence for 'time flies':", tags)
```

Experiment 6: Vector Space Models & TF-IDF Cosine Similarity in Python

Objective: Implement Term Frequency-Inverse Document Frequency (TF-IDF) scoring and compute pairwise Cosine Document Similarity across a text corpus.

```
import numpy as np
import math

def compute_tfidf(docs):
    vocab = sorted(list(set(w for d in docs for w in d.lower().split())))
    n_docs = len(docs)
    tfidf_matrix = []

    # Document Frequency (DF)
    df = {w: sum(1 for d in docs if w in d.lower().split()) for w in vocab}

    for d in docs:
        tokens = d.lower().split()
        tf = Counter(tokens)
        row = []
        for w in vocab:
            tf_val = tf[w] / len(tokens)
            idf_val = math.log((1 + n_docs) / (1 + df[w])) + 1
            row.append(tf_val * idf_val)
        tfidf_matrix.append(row)
    return np.array(tfidf_matrix), vocab

docs = [
    "deep learning transforms natural language processing",
    "natural language understanding and machine learning",
    "compiler design and code optimization"
]

matrix, vocab = compute_tfidf(docs)
cos_sim_0_1 = np.dot(matrix[0], matrix[1]) / (np.linalg.norm(matrix[0]) * np.linalg.norm(matrix[1]))
print(f"Cosine Similarity (Doc 0 vs Doc 1): {cos_sim_0_1:.4f}")
```

Experiment 7: Word2Vec Skip-Gram Architecture with Negative Sampling

Objective: Construct and train a Word2Vec Skip-Gram embedding model in PyTorch using binary cross-entropy negative sampling loss ($\mathcal{L}_{\text{NEG}} = -\log \sigma(v'_{w_O} \cdot v_{w_I}) - \sum \log \sigma(-v'_{w_i} \cdot v_{w_I})$).

```
import torch
import torch.nn as nn
import torch.optim as optim

class SkipGramNegativeSampling(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.in_embed = nn.Embedding(vocab_size, embed_dim)
        self.out_embed = nn.Embedding(vocab_size, embed_dim)

    def forward(self, target, context, negative):
        # Target: [batch], Context: [batch], Negative: [batch, num_neg]
        v_target = self.in_embed(target)                    # [batch, embed_dim]
        v_context = self.out_embed(context)                 # [batch, embed_dim]
        v_neg = self.out_embed(negative)                    # [batch, num_neg, embed_dim]

        # Positive loss
        pos_score = torch.sum(v_target * v_context, dim=1)   # [batch]
        pos_loss = -torch.log(torch.sigmoid(pos_score) + 1e-7)

        # Negative loss
        neg_score = torch.bmm(v_neg, v_target.unsqueeze(2)).squeeze(2) # [batch, num_neg]
        neg_loss = -torch.sum(torch.log(torch.sigmoid(-neg_score) + 1e-7), dim=1)

        return torch.mean(pos_loss + neg_loss)
```

Experiment 8: Transformer Scaled Dot-Product Multi-Head Attention Layer

Objective: Implement the multi-head self-attention mechanism $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$ from scratch in PyTorch.

```
import torch
import torch.nn as nn
import math

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model=64, num_heads=4):
        super().__init__()
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        batch_size, seq_len, _ = x.shape
        # Project and split into heads: [batch, num_heads, seq_len, d_k]
        Q = self.W_q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)

        # Scaled Dot-Product Attention
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
        if mask is not None: scores = scores.masked_fill(mask == 0, -1e9)
        attn_weights = torch.softmax(scores, dim=-1)
        context = torch.matmul(attn_weights, V) # [batch, num_heads, seq_len, d_k]

        # Concatenate heads and project out
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.d_model)
        return self.W_o(context), attn_weights

x = torch.randn(2, 8, 64) # Batch=2, Seq=8, d_model=64
mha = MultiHeadSelfAttention(d_model=64, num_heads=4)
out, attn = mha(x)
print(f"Multi-Head Output Shape: {out.shape} | Attention Matrix Shape: {attn.shape}")
```

Experiment 9: Machine Translation Evaluation: BLEU Metric from Scratch

Objective: Implement the bilingual evaluation understudy (BLEU) metric from scratch in Python: modified N-gram precision (p_1, p_2, p_3, p_4) with geometric average and brevity penalty (BP).

```
import math
from collections import Counter

def modified_precision(candidate, references, n=1):
    cand_ngrams = [tuple(candidate[i:i+n]) for i in range(len(candidate) - n + 1)]
    if not cand_ngrams: return 0.0
    cand_counts = Counter(cand_ngrams)

    max_ref_counts = Counter()
    for ref in references:
        ref_ngrams = [tuple(ref[i:i+n]) for i in range(len(ref) - n + 1)]
        ref_counts = Counter(ref_ngrams)
        for g in cand_counts:
            max_ref_counts[g] = max(max_ref_counts[g], ref_counts[g])

    clipped_counts = {g: min(count, max_ref_counts[g]) for g, count in cand_counts.items()}
    return sum(clipped_counts.values()) / sum(cand_counts.values())

def compute_bleu(candidate_str, reference_strs, max_n=4):
    cand = candidate_str.lower().split()
    refs = [r.lower().split() for r in reference_strs]

    c = len(cand)
    r = min(len(ref) for ref in refs) # Effective reference length
    bp = 1.0 if c > r else math.exp(1 - r / c)

    p_ns = [modified_precision(cand, refs, n=i) for i in range(1, max_n + 1)]
    if any(p == 0 for p in p_ns): return 0.0

    log_sum = sum((1.0 / max_n) * math.log(p) for p in p_ns)
    return bp * math.exp(log_sum)

cand = "the cat sat on the mat"
refs = ["the cat was sitting on the mat", "there is a cat on the mat"]
print(f"Computed BLEU-4 Score: {compute_bleu(cand, refs) * 100:.2f}")
```

Experiment 10: Character-Level Recurrent Neural Network (RNN) in PyTorch

Objective: Train a character-level vanilla RNN / LSTM model in PyTorch to learn sequence transitions and autonomously generate synthetic text.

```
import torch
import torch.nn as nn

class CharRNN(nn.Module):
    def __init__(self, vocab_size, hidden_size=64, num_layers=1):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.rnn = nn.RNN(hidden_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, h0=None):
        if h0 is None: h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)
        embedded = self.embed(x)
        out, hn = self.rnn(embedded, h0)
        logits = self.fc(out)
        return logits, hn

# Sample usage with dummy vocab size = 28
model = CharRNN(vocab_size=28, hidden_size=32)
dummy_input = torch.randint(0, 28, (1, 10)) # Batch=1, Seq=10
logits, _ = model(dummy_input)
print("CharRNN Output Logits Shape:", logits.shape)
```

Experiment 11: Sentiment Analysis with TF-IDF & Logistic Regression

Objective: Build a text classification pipeline to classify customer product reviews into Positive (1) and Negative (0) sentiments using TF-IDF features and Sigmoid Cross-Entropy.

```
import numpy as np

def sigmoid(z): return 1.0 / (1.0 + np.exp(-z))

def train_sentiment_model(X_tfidf, y, lr=0.1, epochs=500):
    n_samples, n_features = X_tfidf.shape
    w = np.zeros(n_features)
    b = 0.0

    for _ in range(epochs):
        z = np.dot(X_tfidf, w) + b
        y_pred = sigmoid(z)
        dw = (1 / n_samples) * np.dot(X_tfidf.T, (y_pred - y))
        db = (1 / n_samples) * np.sum(y_pred - y)
        w -= lr * dw
        b -= lr * db
    return w, b

X_mock = np.array([[0.8, 0.2], [0.1, 0.9], [0.7, 0.3], [0.2, 0.8]])
y_mock = np.array([1, 0, 1, 0])
w, b = train_sentiment_model(X_mock, y_mock)
print(f"Trained Sentiment Model Weights: {w}, Bias: {b:.4f}")
```

Experiment 12: Named Entity Recognition (NER) BIO Tagging Pipeline

Objective: Implement BIO (Begin-Inside-Outside) sequence labeling for Named Entity Recognition to extract Person (`PER`), Organization (`ORG`), and Location (`LOC`) entities.

```
def bio_entity_extractor(sentence, tags):
    words = sentence.split()
    entities = []
    current_entity = []
    current_type = None

    for w, t in zip(words, tags):
        if t.startswith('B-'):
            if current_entity:
                entities.append((' '.join(current_entity), current_type))
            current_entity = [w]
            current_type = t[2:]
        elif t.startswith('I-') and current_type == t[2:]:
            current_entity.append(w)
        else:
            if current_entity:
                entities.append((' '.join(current_entity), current_type))
                current_entity = []
                current_type = None
    if current_entity: entities.append((' '.join(current_entity), current_type))
    return entities

sent = "Shaswat Raj studies at Birla Institute of Technology in Mesra"
tags = ["B-PER", "I-PER", "O", "O", "B-ORG", "I-ORG", "I-ORG", "I-ORG", "O", "B-LOC"]
print("Extracted Named Entities:", bio_entity_extractor(sent, tags))
```

Experiment 13: Extractive Text Summarization via TextRank (PageRank) in Python

Objective: Implement an unsupervised graph-based TextRank ranking algorithm using sentence cosine similarity and PageRank power iteration to extract the top-k summary sentences.

```
import numpy as np

def sentence_similarity(sent1, sent2):
    w1, w2 = set(sent1.lower().split()), set(sent2.lower().split())
    if not w1 or not w2: return 0.0
    return len(w1 & w2) / (math.log(len(w1) + 1) + math.log(len(w2) + 1))

def textrank_summarize(sentences, top_k=2, d=0.85, max_iter=50):
    n = len(sentences)
    sim_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i != j: sim_matrix[i][j] = sentence_similarity(sentences[i], sentences[j])

    # Normalize transition matrix
    row_sums = sim_matrix.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1.0
    P = sim_matrix / row_sums

    # PageRank Iteration
    scores = np.ones(n) / n
    for _ in range(max_iter):
        scores = (1 - d) / n + d * np.dot(P.T, scores)

    ranked_indices = np.argsort(scores)[::-1][:top_k]
    return [sentences[i] for i in sorted(ranked_indices)]

doc = [
    "Deep learning has revolutionized artificial intelligence and natural language processing.",
    "Transformers allow machines to understand complex contextual relationships in human text.",
    "Compiler design involves lexical, syntactic, and semantic translation phases.",
    "Modern language models like BERT and GPT utilize self-attention mechanisms for high accuracy."
]
print("TextRank Summary:\n", textrank_summarize(doc, top_k=2))
```

Experiment 14: Word Sense Disambiguation (WSD) via Simplified Lesk Algorithm

Objective: Implement the Simplified Lesk dictionary overlap algorithm in Python to disambiguate polysemous words based on context overlap with gloss definitions.

```
def simplified_lesk(word, sentence, synset_glosses):
    context = set(sentence.lower().split()) - {word.lower()}
    best_sense = None
    max_overlap = -1

    for sense, gloss in synset_glosses.items():
        gloss_words = set(gloss.lower().split())
        overlap = len(context & gloss_words)
        if overlap > max_overlap:
            max_overlap = overlap
            best_sense = sense
    return best_sense, max_overlap

glosses = {
    'bank.n.01': 'sloping land beside a body of water such as a river or lake',
    'bank.n.02': 'a financial institution that accepts deposits and channels money into lending activities'
}
sent = "The company deposited money in the bank"
sense, overlap = simplified_lesk('bank', sent, glosses)
print(f"Disambiguated Sense: {sense} (Context Overlap = {overlap})")
```

Comprehensive Viva-Voce Question Bank & Model Answers

> **Q1. Why is Scaled Dot-Product Attention divided by $\sqrt{d_k}$?**
>
> For large projection dimensions d_k, the dot product $q \cdot k$ grows large in magnitude, pushing the Softmax function into regions with extremely small gradients ($\frac{\partial \text{softmax}}{\partial z} \approx 0$). Scaling by $\frac{1}{\sqrt{d_k}}$ preserves unit variance and prevents vanishing gradients during backpropagation!

Q2. Differentiate between Continuous Bag-of-Words (CBOW) and Skip-Gram in Word2Vec.

• **CBOW:** Predicts the central target word w_t given the surrounding context window words $(w_{t-k}, \dots, w_{t+k})$. Faster to train, higher accuracy on frequent words.
• **Skip-Gram:** Predicts surrounding context words given a single central target word w_t. Slower, but superior performance on rare and infrequent words.

Q3. What is the Brevity Penalty in BLEU and why is it necessary?

Modified N-gram precision alone rewards short candidate sentences (e.g., candidate "the" might get 100% precision). The **Brevity Penalty** $\text{BP} = \min(1, e^{1-r/c})$ severely penalizes translations that are shorter than the reference translation length r.

Q4. Explain the difference between Stemming and Lemmatization.

Stemming (e.g., Porter Stemmer) applies crude, fast rule-based suffix chopping which often produces non-words (e.g., `running` $\rightarrow$ `run`, `studies` $\rightarrow$ `studi`). **Lemmatization** (e.g., WordNet) uses morphological analysis and dictionary vocabulary lookups to return the valid base dictionary lemma (e.g., `better` $\rightarrow$ `good`).

Q5. Explain how Positional Encoding enables Transformers to understand token order.

Because the Transformer self-attention operation is order-invariant (permutation-symmetric), sinusoidal positional encodings $PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$ and $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$ are directly added to token embeddings, allowing the model to distinguish word order without recurrent loops!

Q6. What is the difference between BERT and GPT architectural pre-training objectives?

BERT is a bidirectional Transformer Encoder trained on Masked Language Modeling (MLM 15% random masking) and Next Sentence Prediction (NSP), optimal for classification and NER. **GPT** is an autoregressive causal Transformer Decoder trained on left-to-right causal next-token prediction, optimal for generative completion.